

A mixture-of-experts language model you can see all of

A tiny MoE built from scratch in sw-MLPL, one mechanism at a time, with every router decision, expert load, and byte measured and recorded. Results from the first three sagas, plus an honest early answer about a trained "campus docent".

7,096 parameters

4 experts, top-1 and top-2

live demo on GitHub Pages

everything measured

WHY

Store far more model than you execute

FreeToken-style inference wants a model whose experts live on disk, whose hot experts sit in a cache, whose memorized patterns live in a lookup table, and whose depth comes from reusing one block. Each of those is a kind of sparsity, and each can be isolated at microscope scale.

Sparsity	Avoid paying for	Mechanism
Parameter	many unique layers	recurrence
Compute	every expert on every token	top-k routing, sparse dispatch
Memory lookup	reconstructing common patterns in weights	Engram table
Residency	the whole expert bank in fast memory	expert cache

Large models hide these parts. A model of a few thousand parameters can show them, if every intermediate value is recorded and drawn.

WHAT

The microscope, not just the model

- **MicroMoE**: a one-block transformer over a 52-symbol synthetic domain (arithmetic, sequences, MLPL fragments, prose), then four experts behind a router with a load-balance term, then top-2, then true sparse dispatch, then sixteen low-rank delta experts.
- **Every lesson records itself**: named frames from the training run, annotated what/why/how diagrams, and three measurements: memory, speed, quality.
- **Written in sw-MLPL**, an array language for ML; twenty language findings met on the way, most fixed upstream the same day.
- **A live demo** replays the pinned recordings frame by frame in the browser. It is a playback, not a live computation, and it says so.

Rule of the repository: prefer a shipped feature; build from scratch only when watching it work is the lesson, then show both forms with a parity check.

RESULTS

Sparse capacity works, and experts specialize

STORED CAPACITY

1.86x

the dense model's parameters, for 1.02x the active parameters per token, at the same validation loss (3.76 vs 3.79).

SPECIALIZATION

0.61

mean largest family share per expert under top-1, against 0.25 if routing ignored the task. The router never saw a family label.

SPARSE DISPATCH

4x

fewer expert row evaluations (13,440 to 3,360) with outputs equal to the bit. And slower in the interpreter: dispatch overhead beats tiny matmuls.

Run	Params	Active / token	Val loss	Note
DN01 dense	3,812	2,996	3.79	baseline
MX01 top-1 of 4	7,096	3,064	3.76	specialization 0.61
MX02 top-2 of 4	7,096	4,136	3.92	fits better, generalizes differently
LD01 16 rank-4 deltas	5,060	2,324	3.37	half the bytes of 4 full experts
RM01 recurrent	7,000	7,000	3.40	same block reused; 38% of

RESULTS

Data moves the frontier. Epochs do not.

Training examples	Dense val loss	MoE val loss	Held-out prose exact match
120, 200 epochs	3.48	3.49	0.67
120, 400 epochs	3.89	4.17	0.67
480, 200 epochs	1.66	1.42	1.00
960, 200 epochs	1.11	1.22	1.00 / 0.75

Doubling epochs at 120 examples makes both models worse. Eightfold data cuts validation loss by two thirds. The small mixture wins at 480 examples and loses at 960: at this scale neither architecture dominates, and the difference only shows once there is enough data.

Every claim cites a results row, a diagram, or a finding, and states its limitation. One seed per point; interpreter timings on one laptop.

HOW

One window through the router, stage by stage

MX02 SPECIALIZATION -- which expert serves which task family

WHAT: counts[family, expert] over masked training tokens, row-normalized; score = mean over families of the largest share

MX01 top-1 score 0.61

	e0	e1	e2	e3
arithmetic	0.3	0.03	0.14	0.53
sequence	0.61	0.05	0.12	0.22
mlpl	0.26	0.24	0.07	0.43
prose	0.87	0	0.13	0

MX02 top-2 score 0.42

	e0	e1	e2	e3
arithmetic	0.44	0.03	0.11	0.41
sequence	0.39	0.17	0.36	0.08
mlpl	0.39	0.14	0.18	0.29
prose	0.46	0.19	0.27	0.08

raw counts, MX02 (rows: families)

76 6 20 72

166 70 152 36

53 19 24 40

36 15 21 6

family-blind: every row 0.25

one expert per family: 1.0

reading the map

A column that is bright in one row and dark in the others is an expert that one family owns.

A row that is bright across columns is a family whose tokens are spread over experts (top-2 spreads by construction).

Nothing here was labeled: the router saw hidden states, never the family tag. Specialization is emergent, or absent.

WHY: the plan promised to measure specialization, not assume it; this map is also the target of the Saga 4 router warm start.

WHY: prose and arithmetic have distinct surface statistics and separate first; sequence and mlpl share digits and stay mixed.

HOW: u:top2_mask * mask row, one_hot(tag) outer counts, u:row_normalize, u:specialization_score; MX01 from its fixture.

Observations moe2/family-by-expert and moe2/specialization; the MX01 fixture is regenerated by just moe write.

Router logits, softmax probabilities, a one-hot mask, a gate that keeps the chosen probability (so the router gets a gradient), per-expert loads, the balance term, then dispatch: gather each expert's rows, run it once, scatter back. The live demo steps through exactly these recorded values.

AN HONEST INTERIM RESULT

Does a tiny model beat keyword matching?

Second training area: a "campus docent" that reads a visitor's question and predicts only an intent and a destination on a museum-style campus site. The catalog supplies every word it says, so it cannot invent an exhibit.

Held-out rows	Dense docent (trained)	Keyword matcher (no model)
Destination accuracy	0.925	0.954
Intent accuracy	0.938	0.780
Paraphrases (no alias verbatim)	0.296	0.630
Off-topic rejected	0.30	1.00

Today the trained docent does not add value over a deterministic matcher. The bar is written down: beat the matcher by 20 points on paraphrases or the site keeps the matcher. Word vectors from the campus's own text and the routed experts are next.

CONCLUSIONS

What held up, and what is next

- Top-1 routing nearly doubles stored capacity at the same active cost; experts specialize on their own when the tasks have structure.
- Sparse dispatch is exact and saves work, but not time, until experts are big enough for the kernels to matter.
- Data quantity decides whether architectural differences show at all.
- Reusing one block three times buys quality per parameter (3.42 at the mixture's 7,096), and the router re-decides at every pass.
- An Engram table written from scratch matches the sw-MLPL builtin to the bit; at 90 training rows it memorizes instead of generalizing, alone and composed.
- A measured yardstick beats a claim: the docent has to earn its place against a matcher.

Next: routed experts for the docent, an animated data-flow landscape of every mechanism, then distillation, quantization, an expert cache, and CPU/GPU heterogeneous execution.

Live demo sw-ml-study.github.io/moe-microscope

Code and results github.com/sw-ml-study/moe-microscope **Wiki**
github.com/sw-ml-study/moe-microscope/wiki

Language github.com/sw-ml-study/sw-mlpl **Blog** blog.softwarewrighter.com